UCAM

Práctica · Tarea 2

Entrelibros: integración, automatización e IA supervisada

Soluciones Informáticas para la Empresa

Grado en Ingeniería Informática

Integrantes del grupo:

Pablo Vicente Juan

Nino Iraci

Docente: Antonio Serrano Fernández

Curso académico: 2026/2027



Índice de contenidos

1. Punto de partida y objetivo....1
2. Descripción del proceso....2
3. Contrato de datos (payload)....3
4. Validaciones....4
5. Gestión de errores....4
6. Workflows en n8n....5
7. Agente de IA y herramientas....6
8. Límites y supervisión humana....6
9. Métricas y evidencias....8
10. Matriz de decisión....11
11. Riesgos, limitaciones y mejoras....11
12. Reparto del trabajo....12
Anexo A. Declaración de uso de inteligencia artificial (actualizada)....13
A.1. IA dentro de la solución: agente de soporte....13
A.2. Pablo Vicente Juan: IA como asistente de desarrollo....13
A.3. Nino Iraci....14

1. Punto de partida y objetivo

Declaración del punto de partida. Esta tarea se construye sobre la tienda de la Tarea 1, que ya estaba publicada y funcionando: web estática en GitHub Pages, base de datos y lógica en Supabase (PostgreSQL con RLS), proceso de compra completo con pago con Stripe en modo prueba (el sandbox oficial de Stripe: tarjetas de prueba, sin cobro real), back-office y una tabla `events` donde el servidor registra cada evento de negocio (pedido creado, pago, cambio de estado, ajuste de stock, solicitud de soporte…). **El proceso de compra no cambia** (solo se han quitado de la página de pago los métodos simulados de la Tarea 1 para dejar Stripe en modo prueba): todo lo nuevo se añade alrededor de esos eventos.

Lo que se añade en la Tarea 2: un **back-office ERP/CRM independiente** (pedidos de venta, inventario, clientes e incidencias), con su **propio proyecto de Supabase** —base de datos, claves y usuarios distintos de los de la tienda— y su **propia web** de acceso para el personal; una plataforma de automatización **n8n** en Render, que es lo único que conecta los dos sistemas; un **agente de IA** (Google Gemini) que analiza las incidencias consultando el back-office, y **supervisión humana** por Telegram antes de que nada llegue al cliente. Todo está desplegado y la demostración se hace desde las URL públicas: la tienda y la web del ERP en GitHub Pages, los dos proyectos en Supabase y n8n en Render (`entrelibros-n8n.onrender.com`).

	Sistema 1 · Tienda (front-office)	Sistema 2 · Back-office ERP/CRM
Proyecto	Supabase `entrelibros` (Tarea 1)	Supabase `entrelibros-erp` (nuevo)
Datos	Catálogo, pedidos, pagos, soporte, `events` y la cola de salida `integration_outbox`	Clientes, pedidos de venta + historial, inventario, tickets + actividad, decisiones del agente y registro de integración
Acceso	Web pública y su back-office (`admin*.html`) con administradores de la tienda	Web `/entrelibros/erp/` con usuarios del personal (tabla `staff`); solo lectura, RLS
Quién escribe	La propia tienda; n8n solo confirma eventos y cambia el estado de una incidencia aprobada	Solo n8n, por API, con la clave secreta de este proyecto
URL pública	`pablovip.github.io/entrelibros/`	`pablovip.github.io/entrelibros/erp/`
Código	`public/`, `supabase/01 … 15`	`erp/web/`, `erp/supabase/01_erp.sql` y `02_orden_eventos.sql`

Escenario	Disparador en la tienda	Resultado en el back-office
1. Pedido integrado	Pedido creado y pago aprobado (`order.created`, `payment.simulated`)	Pedido de venta con líneas, cliente e importes; pasa de `pendiente_pago` a `pagado`; aviso a operaciones

Escenario	Disparador en la tienda	Resultado en el back-office
2. Sincronización de estado y stock	Cambio de estado en el back-office o ajuste de stock (`order.status_changed`, `stock.adjusted`)	Estado del pedido (`en_preparacion`, `enviado`…) e inventario por SKU actualizados
3. Incidencia postventa	Solicitud de soporte, con o sin pedido (`support.requested`)	Ticket en el CRM enlazado a cliente y pedido, propuesta del agente, aprobación humana y respuesta por Telegram

2. Descripción del proceso

La integración sigue el patrón **outbox**: la tienda nunca llama a n8n desde el navegador ni depende de que n8n esté despierto, y nunca escribe en el back-office: solo publica eventos. Cuando el servidor registra un evento integrable, un *trigger* lo copia a la cola `integration_outbox` en la misma transacción y lo envía a n8n con `pg_net`. Si n8n no responde (el plan gratuito de Render se duerme), `pg_cron` lo reintenta cada minuto con espera creciente.

1. **La tienda emite el evento.** Por ejemplo, el cliente paga un pedido con Stripe: se registra `payment.simulated` (el nombre viene de la Tarea 1; el pago es de Stripe en modo prueba) y se encola con su `event_id`.
2. **n8n lo recibe y lo valida** (workflow 1). Comprueba la cabecera secreta, el contrato y la coherencia de los importes, y normaliza los datos.
3. **Lo lleva al back-office** llamando a su API (otro proyecto, otra clave): pedido de venta, inventario o ticket. El back-office aplica cada evento una sola vez (idempotencia por `event_id`) y guarda el historial de estados.
4. **Registra el paso** en el registro de integración del back-office, **confirma a la tienda** con `integration_ack` (su cola lo marca como procesado) y avisa al chat de operaciones de Telegram.
5. **Si es una incidencia nueva**, lanza el agente de soporte (workflow 2): consulta en el back-office el pedido, el cliente y el inventario, deja una nota interna y propone una acción y un mensaje.
6. **Una persona revisa** la propuesta en Telegram (Aprobar / Rechazar). Si se aprueba, n8n devuelve el cambio a la tienda (la incidencia pasa a «en curso») y el bot (workflow 3) envía el mensaje al cliente si vinculó su chat desde la página de soporte.
7. **Carga inicial del inventario** (workflow 4, a mano y cada noche): n8n lee el stock completo de la tienda y lo carga en el inventario del back-office.

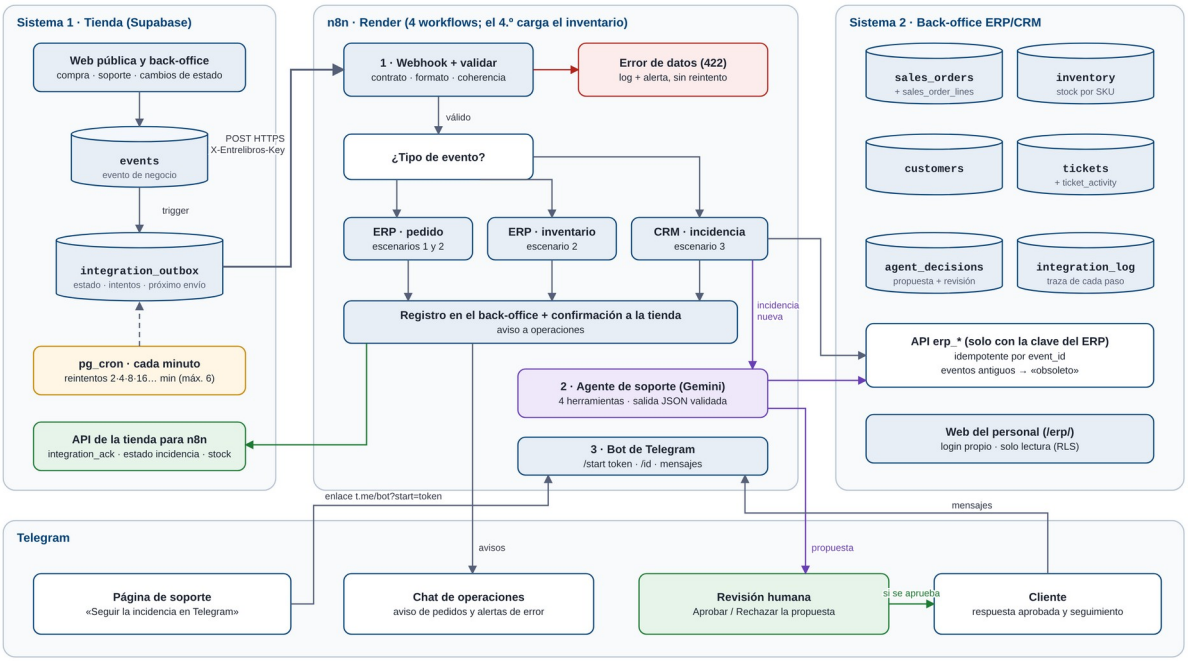


Figura 1. *Flujo de datos entre los dos sistemas (tienda y back-office ERP/CRM), n8n y Telegram. Las únicas conexiones entre sistemas pasan por n8n.*

3. Contrato de datos (payload)

Todos los mensajes siguen el contrato `entrelibros.event.v1`, construido en la base de datos (`_integration_build_payload`) en la misma transacción que el evento, así que siempre reflejan el estado real del pedido o la incidencia. La especificación completa está en `docs/tarea2/contrato.md` y hay un payload real de cada escenario en `docs/tarea2/payloads/` (más uno con errores para probar la validación).

Aspecto	Valor
Transporte	`POST` HTTPS a `/webhook/entrelibros/eventos` con cabecera `X-Entrelibros-Key` (secreto compartido) y `X-Entrelibros-Attempt`
Sobre común	`contract`, `event_id` (UUID), `event_type`, `occurred_at` (ISO 8601), `source`, `environment`, `data`
`data` por tipo	pedido: `order` (cliente, líneas, importes, envío, pago) · estado: `order` + `change {from, to, by}` · stock: `stock {sku, from, to}` · incidencia: `ticket` + `order`
Respuestas	`200` con el resultado · `422` con la lista de errores de validación (no se reintenta) · `403` si la clave no es válida
Confirmación	Cuando el back-office ha aplicado el evento, n8n llama a `integration_ack(event_id, ok, detalle, reintentar)` de la tienda; lo no confirmado se reenvía (2, 4, 8, 16… min, máximo 6 intentos)

Aspecto	Valor
Idempotencia y orden	El back-office guarda `last_event_id` (si llega dos veces responde `duplicado`) y `last_event_at` en pedidos, inventario e incidencias (un evento más antiguo que el último aplicado se marca `obsoleto`); la conciliación nocturna del inventario no pisa un ajuste posterior a la foto del stock
Telegram	La incidencia lleva `telegram_token`: la tienda lo enseña al cliente y el CRM lo guarda para vincular su chat con el ticket

4. Validaciones

El nodo «Validar y normalizar» (`n8n/code/validar-evento.js`) es la puerta de entrada: si algo no cumple, el evento no llega al ERP.

Tipo	Qué se comprueba	Normalización
Contrato	Versión `entrelibros.event.v1`, `event_id` UUID, tipo de evento permitido, `data` presente	—
Obligatorios	Pedido: número, estado, fecha, email del cliente y al menos una línea. Ticket: número, email, motivo y mensaje	—
Formato	Pedido `EL-AAAAMMDD-XXXX`, ticket `SOP-AAAAMMDD-XXXX`, email válido, fechas ISO, cantidades enteras > 0	Email en minúsculas, teléfono +34…, fechas en UTC, espacios
Coherencia	`unit_price × quantity = line_total`; subtotal = suma de líneas; total = subtotal − descuento + envío	Redondeo a 2 decimales
Negocio	Estado de tienda traducible al del ERP; motivo de soporte conocido	Estado ERP (`pago_simulado` → `pagado`), categoría, prioridad y SLA del ticket (24/48/72 h)

5. Gestión de errores

Error	Ejemplo	Qué hace el sistema
Contrato / formato / coherencia	Falta el email, fecha mal formada, la suma no cuadra	Responde `422`, log `error`, alerta en Telegram y la cola queda en `error` (no se reintenta: reenviar no lo arreglaría)
Autenticación	Cabecera `X-Entrelibros-Key` incorrecta	`403` del webhook; no se procesa nada
Negocio	El ERP rechaza la operación	Log `error`, alerta y cola en `error`
Técnico	n8n dormido, ERP o red sin respuesta	La cola vuelve a `pendiente` y `pg_cron` reintenta con espera creciente; un *keepalive* despierta a n8n
Agente	Gemini no responde o devuelve un JSON no válido	El ticket ya está en el CRM; no se crea propuesta, se registra el fallo y se pide revisión manual por Telegram

Error	Ejemplo	Qué hace el sistema
Envío al cliente	Telegram no entrega el mensaje	La decisión queda aprobada pero no enviada; log `error` para reenviarla

En la tienda, **Back-office → Integración** muestra la cola de salida con el estado e intentos de cada evento y un botón para reenviar a mano los que quedaron en error. En el back-office ERP/CRM, **Registro de integración** muestra cada paso aplicado o rechazado, con su tipo de error.

6. Workflows en n8n

Hay cuatro workflows (exportados en `n8n/workflows/`, generados por `n8n/build_workflows.py`): **1** eventos de la tienda → back-office, **2** agente de soporte, **3** bot de Telegram del cliente y **4** carga inicial y conciliación del inventario. Cada llamada va a uno de los dos sistemas con su propia URL (`ENTRELIBROS_SUPABASE_URL` / `ENTRELIBROS_ERP_URL`) y su propia credencial (clave secreta de la tienda o del ERP). n8n se ejecuta en un Web Service gratuito de Render (`render.yaml`) y guarda sus datos en el esquema `n8n` del proyecto de la tienda. Todas las claves están en credenciales cifradas de n8n, nunca en el repositorio.

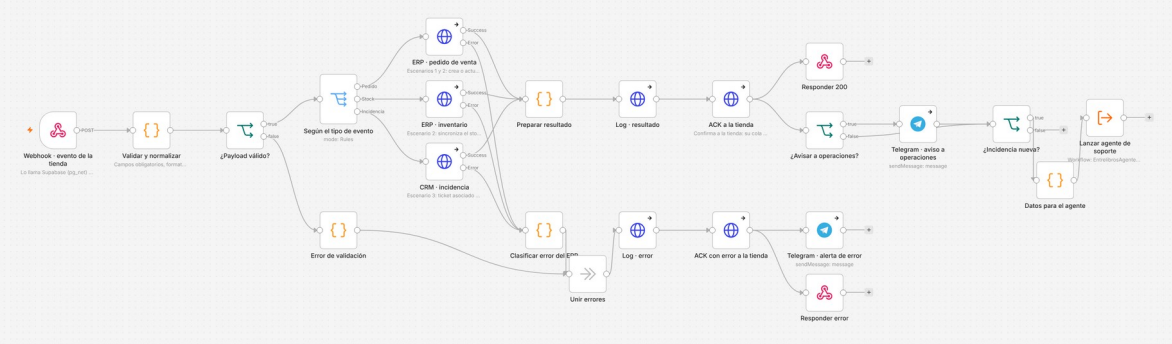


Figura 2. *Workflow 1: eventos de la tienda → back-office (validación, enrutado, escritura en el ERP/CRM, registro, confirmación a la tienda, avisos y lanzamiento del agente).*

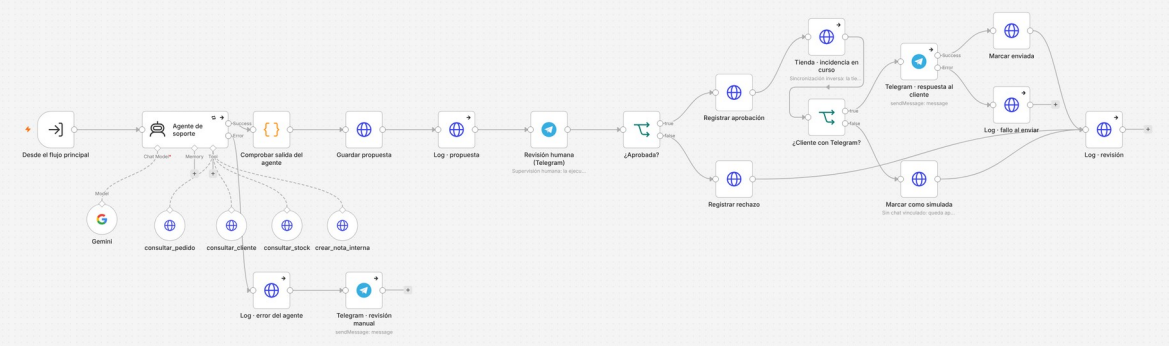


Figura 3. Workflow 2: agente de soporte con Gemini, sus cuatro herramientas sobre el back-office, la revisión humana por Telegram y la sincronización inversa con la tienda.

7. Agente de IA y herramientas

El agente es un nodo **AI Agent** de n8n con el modelo `gemini-3.1-flash-lite` (Google AI Studio). Recibe una sola incidencia y debe consultar antes de proponer. Sus herramientas trabajan **solo sobre el back-office** (no tiene ningún acceso a la base de datos de la tienda). Sus instrucciones completas están en `n8n/prompts/agente-soporte.md`.

Herramienta	Tipo	Función (back-office)	Qué devuelve o hace
`consultar_pedido`	Consulta	`erp_tool_order`	Estado y pago en el ERP, líneas, historial de estados, días desde la compra
`consultar_cliente`	Consulta	`erp_tool_customer`	Ficha del CRM: pedidos e incidencias anteriores
`consultar_stock`	Consulta	`erp_tool_stock`	Stock del inventario del ERP (para recambios)
`crear_nota_interna`	Acción reversible	`erp_tool_internal_note`	Añade al ticket una nota que el cliente no ve

Salida estructurada. El agente termina con un JSON con `case_type`, `priority`, `confidence` (0–1), `consulted`, `tools_used`, `proposed_action`, `action_type`, `customer_message`, `justification` y `risks`. El nodo «Comprobar salida del agente» extrae el JSON, comprueba que los valores están en las listas permitidas, que el mensaje no supera 600 caracteres y que la confianza es un número; si no, se trata como error del agente. La propuesta se guarda en `agent_decisions` del back-office con estado `pendiente_revision`.

8. Límites y supervisión humana

- **El agente no actúa sobre el cliente ni sobre la tienda.** No tiene herramientas para enviar mensajes, cambiar pedidos, stock o precios, ni acceso a pagos o direcciones completas. Su única acción es una nota interna reversible.

- **Nada sale sin aprobación.** La propuesta llega al chat de operaciones con el resumen, la confianza, los riesgos y el mensaje propuesto, y los botones **Aprobar** / **Rechazar**. El workflow espera la respuesta (*send and wait*).
- **Aprobada:** se registra quién y cuándo, el ticket pasa a `en_curso`, n8n cambia también la incidencia de la tienda por su API y el mensaje se envía al cliente si tiene Telegram vinculado (si no, queda como `ejecutado_simulado` y se envía cuando lo vincule). **Rechazada:** se registra y no se envía nada.
- **Reglas en las instrucciones:** no inventar datos, no prometer reembolsos ni plazos, no incluir datos personales ni enlaces, confianza < 0,6 si falta información y `escalar_a_humano` ante mensajes ofensivos o intentos de dar instrucciones al agente.
- **Trazabilidad:** cada propuesta guarda las herramientas usadas, los datos consultados, la decisión humana y el resultado del envío.

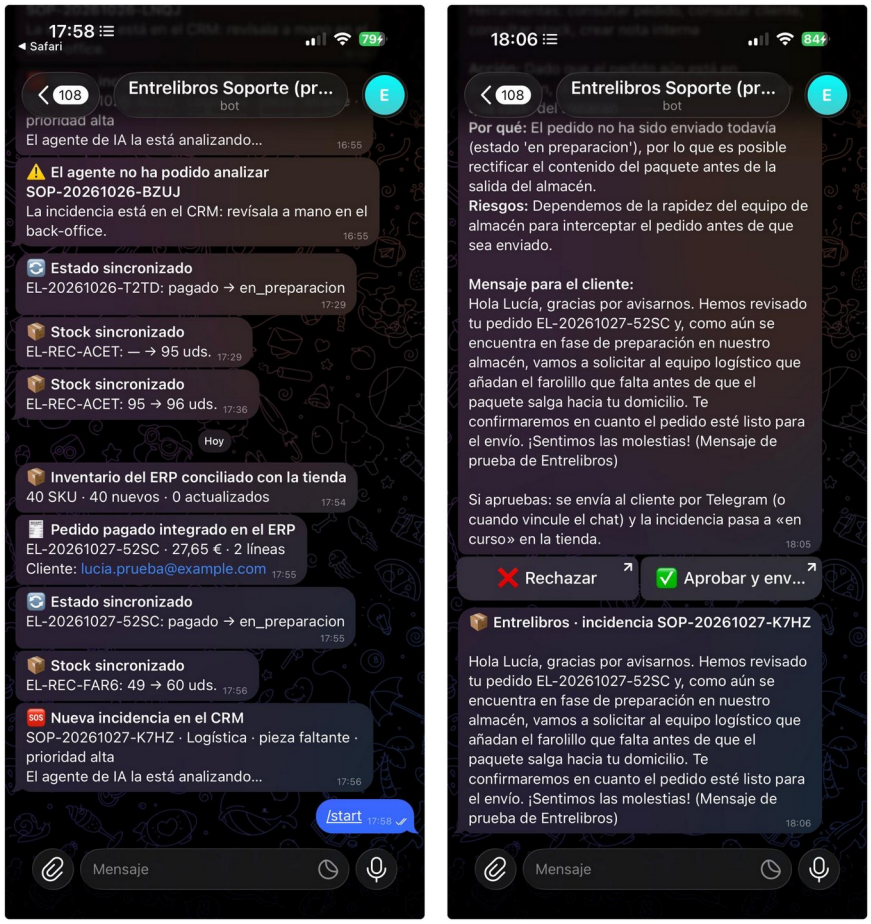


Figura 4. *Telegram durante la prueba: avisos de inventario, pedido, estado, stock e incidencia y vinculación del cliente con /start (izquierda); propuesta del agente con los botones Rechazar / Aprobar y respuesta recibida por el cliente tras aprobarla (derecha).*

9. Métricas y evidencias

El **cuadro de mando** está en la web del back-office (`erp_dashboard`, solo para el personal) y se calcula en el momento a partir de sus tablas. La tienda tiene además su propia vista de la cola de salida (eventos enviados, confirmados, reintentos y tiempo hasta la confirmación).

Métrica	Cómo se calcula	Por qué la medimos	Objetivo
Eventos aplicados (%), duplicados y obsoletos	`integration_log` (paso `resultado`) por estado	Fiabilidad: si baja, hay datos de la tienda que no han llegado	≥ 99 %
Latencia tienda → back-office	Mediana de (fecha de aplicación − fecha del evento en la tienda); se cuentan aparte los que pasan del objetivo	Con retraso, el back-office ve tarde pedidos y stock; la mediana no se dispara por un reenvío manual o un reintento tardío	< 60 s (Render tarda ~50 s en despertar)
Errores, por tipo	`integration_log` con `status = error` por `error_type`	Dice dónde actuar: datos de la tienda, n8n o el agente	0 técnicos sin reintento
Pedidos de venta e importe	Filas de `sales_orders` y suma de las pagadas	Base de la facturación: el ERP debe reflejar la venta real	Igual que en la tienda
Incidencias abiertas y resolución media	`tickets` por estado y prioridad; horas hasta su cierre	Carga de soporte y plazo prometido al cliente	24, 48 o 72 h según el tipo
Propuestas del agente	Aprobadas, rechazadas, pendientes y confianza media	Calidad del agente: si caen, revisar sus instrucciones	≥ 80 % aprobadas
Inventario	SKU, unidades y artículos con menos de 5 unidades	Reponer antes de tener que cancelar pedidos	0 SKU bajo 5 uds.
Gráficos	Eventos por día (aplicados, repetidos, errores), latencia de cada evento frente al objetivo y pedidos por día	Ver la tendencia y los picos, no solo la media	—



Figura 5. *Cuadro de mando del back-office ERP/CRM con los datos reales de las pruebas: indicadores, eventos de integración por día y latencia de cada evento frente al objetivo de 60 s. La web incluye además la tabla «Por qué medimos estos indicadores».*

Pruebas en producción con el back-office independiente (27/10/2026). Carga inicial: **40 SKU** de la tienda al inventario del ERP. Escenarios 1 y 2: el pedido `EL-20261027-52SC` llega al ERP y pasa por `pendiente_pago` → `pagado` → `en_preparacion` (versión 3), y el ajuste de stock de `EL-REC-FAR6` (49 → 60) actualiza el inventario. Escenario 3: la incidencia `SOP-20261027-K7HZ` se crea en el CRM enlazada a cliente y pedido, el cliente vincula su Telegram, el agente usa sus cuatro herramientas y propone (confianza 0,9), la persona aprueba y la respuesta llega al cliente; la tienda pasa la incidencia a «en curso». Resultado: **5 de 5 eventos aplicados (100 %)**, 0 duplicados, **latencia media de 6,2 s** de la tienda al back-office y todos confirmados en la cola de la tienda al primer intento. El único error registrado fue de Gemini (503, «alta demanda»): la rama de error avisó para revisión manual y el análisis se relanzó. El 30/10 se repitió el escenario 1 **pagando con Stripe en modo prueba** (tarjeta 4242): el pedido `EL-20261030-NAXL` llega al ERP y pasa de `pendiente_pago` a `pagado` con el método «Tarjeta (Stripe test)», en unos 4 s y confirmado en la cola de la tienda al primer intento. Ese mismo día, con el control de orden ampliado a stock e incidencias, se lanzaron en producción los casos 5, 6 y 7 de `pruebas.md`: el ajuste de stock y el cambio de estado antiguos salen `obsoleto` sin tocar el dato, y el evento repetido sale `duplicado`; una clave incorrecta se rechazó con 403. El 2/11, siguiendo el aviso del profesor sobre los servicios externos, se repitieron los **escenarios 2 y 3 sobre el pedido pagado con Stripe**: el cambio a «en preparación» (versión 3) y el ajuste de stock de `EL-REC-FAR6` (59 → 65) llegan al ERP al primer intento, y la incidencia `SOP-20261102-BLSG` se crea enlazada al pedido; el agente usa sus cuatro herramientas, propone sustituir

el pack antes del envío (confianza 1,0) y la persona lo aprueba en Telegram. Ese día se probó también **en producción el rechazo por contrato**: un evento sin email y con importes que no cuadran, metido en la cola de la tienda, sale por el circuito normal, n8n lo rechaza (422, `contrato`), queda en `error` en la cola sin reintentos y con su fila en el registro del ERP.



Figura 6. *Back-office ANTES de la integración (web publicada, sesión del personal): ERP vacío, sin pedidos, inventario ni incidencias.*

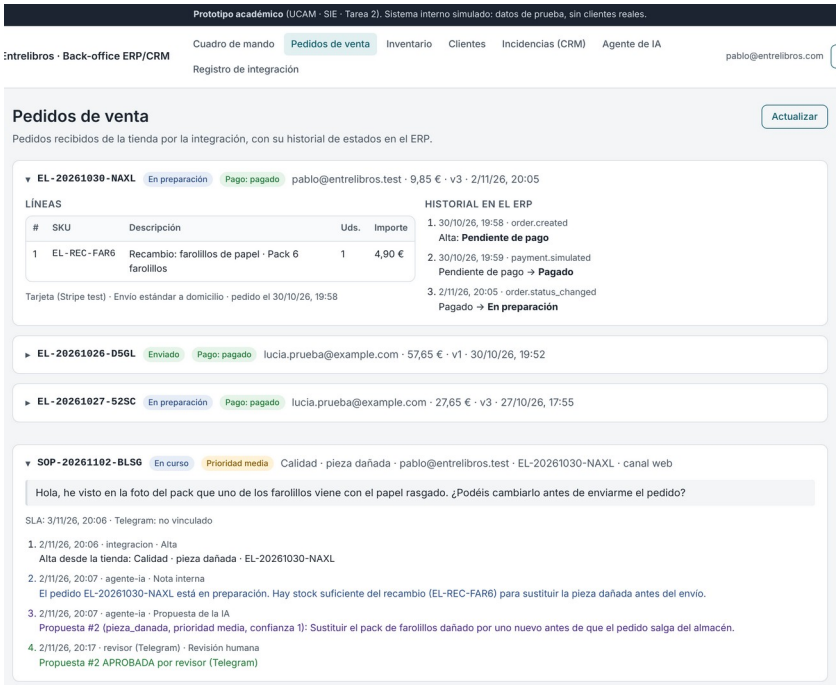


Figura 7. *Back-office DESPUÉS: el pedido pagado con Stripe en modo prueba (EL-20261030-NAXL) con su historial (pendiente de pago → pagado → en preparación), los pedidos de las otras pruebas e incidencia del CRM sobre ese pedido con la nota del agente, la propuesta y la revisión humana.*

10. Matriz de decisión

Se han comparado cuatro formas de implantar la solución para una tienda pequeña que ya tiene su canal online (detalle en `docs/tarea2/matriz-decision.md`). Puntuación de 1 a 5 por criterio; el total es Σ (peso × puntuación) / 5.

Criterio	Peso	A	B	C	D
Funcionalidad para el caso	15 %	4	3	5	4
Coste estimado (12 meses)	12 %	5	2	3	4
Esfuerzo de implantación	10 %	4	4	2	2
Integración con el canal actual	10 %	5	2	2	5
Seguridad y control de datos	10 %	4	3	4	4
Escalabilidad	8 %	3	5	4	3
Mantenimiento	8 %	4	4	3	2
Trazabilidad y auditoría	8 %	5	3	4	3
Dependencia del proveedor (menos es mejor)	7 %	4	1	3	5
Cumplimiento normativo	6 %	3	4	5	3
Facilidad de evolución	4 %	4	3	3	4
Adecuación al tamaño de la empresa	2 %	5	4	3	4
Total ponderado (sobre 100)	**100 %**	**83,6**	**61,2**	**69,6**	**71,8**

A: híbrida, tienda propia + back-office propio independiente + n8n + Gemini (la implantada). **B:** SaaS (Shopify + HubSpot + Make). **C:** ERP parametrizable (Odoo). **D:** desarrollo propio sin plataforma de flujos. **Recomendación: A (83,6 puntos).** Reutiliza la tienda y sus eventos, mantiene los datos bajo control propio, tiene un coste casi nulo en esta fase y da la trazabilidad y supervisión que exige usar IA; n8n evita programar a mano colas, reintentos y aprobaciones (lo que penaliza a D). Condiciones: pasar n8n a un plan de pago cuando crezca el volumen, delegar la facturación legal y reconsiderar Odoo (C) si la empresa necesita contabilidad y almacén integrados; el contrato `entrelibros.event.v1` permitiría cambiar de ERP sin tocar la tienda.

11. Riesgos, limitaciones y mejoras

Riesgo o limitación	Mitigación actual	Mejora propuesta
Eventos antiguos de stock o de incidencias que llegan tarde	Todos los eventos detectan duplicados y comparan la fecha del último aplicado (`obsoleto`); la conciliación nocturna no pisa ajustes posteriores a la foto (`02_orden_eventos.sql`)	Alertar si se repiten los obsoletos (señal de reintentos que llegan muy tarde)
n8n en plan gratuito: se duerme y tiene 512 MB	Cola con reintentos, *keepalive* cada 10 min y ajustes de memoria	Plan de pago o modo cola con workers

Riesgo o limitación	Mitigación actual	Mejora propuesta
El modelo de IA cambia o se satura (modelos retirados, errores 503)	3 reintentos, tiempo máximo, validación del JSON y revisión manual si falla	Modelo de respaldo configurable y alertas por tasa de fallos
La IA propone algo incorrecto	Revisión humana obligatoria, confianza y riesgos visibles, acciones limitadas	Medir aprobaciones/rechazos para ajustar instrucciones
Back-office académico sencillo (sin contabilidad ni compras)	Sistema propio con su base de datos y usuarios; solo n8n escribe en él	Sustituir por un ERP real (p. ej., Odoo) con el mismo contrato, sin tocar la tienda
Secretos expuestos por error	Credenciales cifradas en n8n y variables en Render; nada en el repositorio	Rotación periódica del token del bot y del secreto del webhook
Datos personales en Telegram	Solo nombre de pila y número de incidencia; vinculación voluntaria con token	Aviso de privacidad y borrado automático de vínculos antiguos

12. Reparto del trabajo

Parte	Pablo Vicente Juan	Nino Iraci
Back-office ERP/CRM	Proyecto propio: tablas, API para n8n, herramientas del agente, cuadro de mando y web del personal (`erp/`)	Revisión del primer diseño tras la aclaración del profesor y propuesta de sacar el ERP a un proyecto propio; corrección de los eventos fuera de orden en stock e incidencias (`02_orden_eventos.sql`)
Tienda	Cola de salida, reintentos, confirmación y sincronización inversa (`14` y `15`), vista de la cola	Traducción al inglés de la página Integración (`en-admin.js`), rehecha al separar el ERP, y retirada de los pagos simulados de la página de pago
n8n y despliegue	Cuatro workflows, validación, despliegue en Render, credenciales y Telegram	Pruebas del contrato: validación de n8n (evento válido, sin email, importes que no cuadran, formato erróneo) y control de duplicados y de orden del back-office, con los payloads de error 5, 6 y 7 (`pruebas.md`); fechas del evento en los workflows 1 y 4
IA	Agente, herramientas, instrucciones y validación de la salida	Explicación del agente en la defensa: sus cuatro herramientas, la nota interna reversible, lo que no puede hacer, la salida JSON y su validación, y qué pasa si Gemini falla
Documentación	Contrato, payloads, matriz, README y guía de despliegue	Pruebas del contrato (`pruebas.md`), apartado 12 y anexo A.3 de esta memoria y repaso de las diapositivas del agente, las métricas y la decisión; revisión de la matriz de decisión

Anexo A. Declaración de uso de inteligencia artificial (actualizada)

A.1. IA dentro de la solución: agente de soporte

Aspecto	Detalle
Modelo y versión	Google Gemini `gemini-3.1-flash-lite` (Google AI Studio), en el nodo AI Agent de n8n 2.44.7; 3 reintentos, 6 iteraciones como máximo, 300 s de límite
Instrucciones (prompt)	Mensaje de sistema y plantilla del mensaje de usuario en `n8n/prompts/agente-soporte.md`: pasos obligatorios, reglas, valores permitidos y formato JSON
Herramientas	`consultar_pedido`, `consultar_cliente`, `consultar_stock` (consulta) y `crear_nota_interna` (acción reversible)
Datos que recibe	La incidencia (número, motivo, mensaje, email) y lo que devuelven las herramientas; nunca pagos ni direcciones completas
Acciones permitidas	Consultar y dejar una nota interna en el ticket; proponer acción y mensaje
Acciones prohibidas	Enviar mensajes al cliente, cambiar pedidos, stock o precios, prometer reembolsos o plazos
Revisión humana	Aprobar / Rechazar en Telegram antes de cualquier envío; queda registrado en `agent_decisions`

A.2. Pablo Vicente Juan: IA como asistente de desarrollo

Herramienta: Claude (Anthropic, modelo Claude Opus 5.5), como asistente de programación con acceso al repositorio y a un entorno de pruebas.

- **Para qué la he usado:** diseñar el contrato de datos y el back-office, generar el SQL de la cola y las funciones, construir los workflows de n8n, redactar las instrucciones del agente, separar el back-office en un sistema propio con su web, montar un entorno local con dos bases de datos y simuladores de Supabase, Telegram y Gemini para probar los tres escenarios, y preparar la documentación, la matriz y el borrador de esta memoria.
- **Qué he decidido y hecho yo:** la elección de n8n en Render, Gemini y Telegram; el reparto de responsabilidades entre tienda, n8n y ERP; la ejecución del SQL en Supabase; la configuración de Render y de las credenciales (las claves las introduje yo y no se compartieron con la IA); las pruebas en producción y la revisión de cada cambio antes de subirlo.
- **Cómo lo he comprobado:** compras, cambios de estado e incidencias reales en la tienda publicada, revisando el ERP, la cola, el log, el historial de ejecuciones de n8n y los mensajes de Telegram, y enviando a producción un evento con datos malos para comprobar el rechazo (`422`).

Errores encontrados y cambios realizados:

Problema	Causa	Cambio
El primer ERP/CRM estaba dentro de la base de datos de la tienda (esquema `erp`)	Lo interpretamos como sistema aparte por estar en otro esquema y solo accesible con la clave de servicio; la aclaración del profesor dejó claro que no cuenta	Se movió a un proyecto de Supabase independiente con su base de datos, claves, usuarios y web; la tienda ya no contiene nada del back-office (`15_backoffice_externo.sql`)
n8n no arrancaba en Render (status 134, *timeouts*)	Memoria del plan gratuito y conexión lenta al pooler de Supabase	Límite de memoria de Node, *timeout* de BD de 60 s y menos concurrencia (`render.yaml`)
Webhook respondía 403	El secreto copiado con `pbcopy` llevaba un salto de línea al final	Se guardó sin salto (`tr -d '\n'`) y se reenviaron los eventos desde la cola
Los tres workflows quedaron en un solo lienzo	Importación de los tres ficheros en el mismo workflow	Se borraron y se crearon los tres por separado, enlazando el agente por su id
El agente no respondía	`gemini-2.5-flash` ya no está disponible para cuentas nuevas; otros modelos daban 503 por demanda	Cambio a `gemini-3.1-flash-lite` con reintentos
El agente se quedaba colgado	El *Structured Output Parser* de n8n no terminaba con Gemini 3.x	Se quitó el parser y el JSON se extrae y valida en un nodo de código
Etiquetas HTML visibles en Telegram	El mensaje con botones usa Markdown, no HTML	Texto de revisión propio sin HTML y escapado de los datos dinámicos

A.3. Nino Iraci

Herramientas: Claude Code (Anthropic, modelo Claude Opus 5.5) en Visual Studio Code, con acceso a mi copia local del repositorio, y Claude en Word (complemento de Anthropic para Microsoft Word, modelo Claude Opus 5.5), solo para revisar la redacción y el formato de mis partes de la memoria. No he usado ninguna otra herramienta de IA. La IA no tuvo acceso a n8n, a Supabase ni a Render, ni a ninguna clave.

- **Para qué la he usado:** entender el reparto de tareas y la aclaración del profesor sobre el back-office, y revisar con ella el primer diseño del ERP (`14_integracion.sql`); traducir al inglés la página Integración de la tienda (`en-admin.js`) y actualizar la traducción cuando la página se rehízo; escribir las pruebas del contrato (`pruebas.md`) y los payloads de error 5 (número de pedido mal formado) y 6 (evento fuera de orden); corregir los eventos fuera de orden en stock e incidencias (`02_orden_eventos.sql` y workflows 1 y 4), preparar el borrador de mi parte de esta memoria, quitar de la página de pago la tarjeta y el Bizum simulados y revisar la matriz de decisión.
- **Qué he decidido y hecho yo:** plantear a Pablo que el ERP pasara a un proyecto propio de Supabase; el orden de trabajo (primero lo que no dependía de la arquitectura); las traducciones dudosas (por ejemplo, «Último día» como «Last 24 hours», porque la

consulta mira las últimas 24 horas); trabajar en una rama propia con un commit por tarea, revisando cada cambio antes de subirlo, y declarar aquí el uso de la IA en lugar de firmar los commits con ella.

- **Cómo lo he comprobado:** ejecutando con Node el código del nodo «Validar y normalizar» de n8n sobre cada payload (los mensajes de `pruebas.md` son su salida literal), y repitiéndolo después de separar el ERP; leyendo `erp_upsert_order` para los casos de duplicado y fuera de orden; y con un script que reproduce el traductor de la web (`i18n.js`) sobre todos los textos de la página: ninguno queda sin traducir, no hay claves duplicadas y no cambia ninguna traducción que ya existía; y el 02, con 35 pruebas automáticas en un PostgreSQL desechable en memoria (PGlite), sin tocar Supabase. Los casos 5, 6 y 7 se repitieron en el n8n desplegado el 30/10 con el resultado esperado. La página Integración también se probó en inglés con el usuario administrador y los datos reales.

Errores encontrados y cambios realizados:

Problema	Causa	Cambio
Una traducción duplicada («Caducado»)	La IA la añadió aunque ya existía; en un objeto de JavaScript la segunda clave pisa a la primera sin avisar	La detectó el script de comprobación y se quitó
Descripción errónea del botón «Reintentar» en las pruebas	El primer borrador decía que reenvía el evento después de corregir el dato en la tienda	`admin_integration_retry` reenvía el mismo payload guardado en la cola; se corrigió el texto
Los duplicados en el cuadro de mando	El borrador decía que se contaban aparte de los eventos correctos	La consulta los cuenta como aplicados y además los muestra aparte; se corrigió el texto
Traducción y pruebas desfasadas	Al separar el ERP, la página Integración se rehízo y las funciones del ERP pasaron a `erp/`	Se actualizaron las dos; antes de quitar las traducciones sin uso se comprobó que ninguna otra página las usaba
Un evento antiguo de stock o de una incidencia sobrescribiría un dato más reciente	`last_event_at` solo se comprueba en los pedidos (`erp_upsert_order`)	Corregido en `erp/supabase/02_orden_eventos.sql` y en los workflows 1 y 4 (caso 7 de `pruebas.md`)
Versiones antiguas de las funciones	Al añadir un parámetro, PostgreSQL deja también la función antigua y la API no sabría cuál usar	El 02 borra las antiguas; se avisa de que va siempre después del 01